

TP – Découvrir Git

Versionner son travail et collaborer sur un projet

BTS CIEL – 1^{re} année

Gestion de projet

VERSION CORRIGÉE – DOCUMENT ENSEIGNANT

Durée	4 h : parties 0 à 8 (≈ 3 h), parties 9 et 10 (≈ 1 h, éventuellement en séance suivante)
Prérequis	ouvrir un terminal, utiliser <code>cd</code> , <code>ls</code> , <code>mkdir</code> , <code>cat</code> , éditer un fichier avec <code>nano</code>
Matériel	poste avec Git ≥ 2.28 (Git Bash sous Windows) ; compte GitHub ou GitLab pour la partie 9

Objectifs. À la fin de ce TP, vous saurez :

- expliquer l'intérêt d'un gestionnaire de versions et distinguer Git de GitHub ;
- configurer Git, créer un dépôt, enregistrer des modifications et consulter l'historique ;
- annuler une erreur sans panique ;
- travailler avec des branches et résoudre un conflit ;
- synchroniser votre travail avec un dépôt distant partagé.

Conventions. Les cadres gris contiennent des commandes à taper dans le terminal, ligne par ligne. Les cadres à barre bleue montrent ce que Git affiche. Selon la langue du système, les messages de Git peuvent apparaître en anglais ou en français.

1 Pourquoi un gestionnaire de versions ?

Imaginez votre projet de 2^e année (épreuve E6) : trois ou quatre étudiants travaillent sur les mêmes fichiers (code, documentation, configurations d'équipements). Sans outil adapté, on voit vite apparaître ceci :

```
| rapport.docx   rapport_v2.docx   rapport_v2_final.docx   rapport_v2_final_OK_bis.docx
```

et des questions auxquelles plus personne ne sait répondre : *qui* a modifié ce fichier, *quand* et *pourquoi* ? Comment revenir à la version qui fonctionnait hier ? Comment réunir le travail de deux personnes sans écraser celui de l'une d'elles ?

Un **gestionnaire de versions** (*VCS*, *Version Control System*) enregistre l'historique complet des modifications d'un ensemble de fichiers. **Git** est aujourd'hui le plus utilisé. Il a été créé en 2005 par Linus Torvalds pour le développement du noyau Linux. Il est *décentralisé* : chaque copie d'un dépôt contient tout l'historique, et l'on peut travailler sans connexion.

Git n'est pas GitHub

Git est un *logiciel* installé sur votre poste. **GitHub**, **GitLab** ou **Gitea** sont des *services d'hébergement* de dépôts Git (sites web) qui ajoutent la gestion des collaborateurs, des tickets, etc. On peut utiliser Git sans jamais utiliser GitHub.

Vocabulaire essentiel

Dépôt (<i>repository</i>)	Dossier suivi par Git. L'historique est stocké dans le sous-dossier caché <code>.git</code> .
Commit	Instantané enregistré de l'état des fichiers, avec un auteur, une date, un message et un identifiant unique.
Index (<i>staging area</i>)	Zone de préparation : ce que l'on a choisi de mettre dans le prochain commit.
Branche	Ligne de développement indépendante. Techniquement, un simple pointeur vers un commit.
HEAD	Indique où l'on se trouve : en général, la branche courante.
Dépôt distant (<i>remote</i>)	Copie du dépôt hébergée sur un serveur, habituellement nommée <code>origin</code> .

Exercice 1 – Réflexion (5 min)

1. Citez deux situations de votre vie d'étudiant où un historique des versions vous aurait été utile.
2. Pour chaque affirmation, indiquez s'il s'agit de **Git** ou de **GitHub** :
 - a) s'installe et s'exécute sur votre poste :
 - b) site web qui héberge des dépôts et gère les droits des collaborateurs :
 - c) fonctionne sans connexion Internet :
 - d) appartient à Microsoft depuis 2018 :

Correction

1. Réponses libres, par exemple : retrouver la version d'un compte rendu avant une mauvaise modification, savoir qui a changé la configuration d'un routeur dans un projet de groupe, fusionner les parties d'un rapport rédigées par plusieurs personnes.
2. a) Git b) GitHub c) Git d) GitHub.

2 Installer et configurer Git

Vérifiez d'abord que Git est installé :

```
git --version
```

S'il ne l'est pas : sous Windows, installez *Git for Windows* (<https://git-scm.com>) puis utilisez le terminal **Git Bash**. Sous Debian ou Ubuntu : `sudo apt install git`.

Avant le premier commit, Git doit savoir *qui vous êtes*. Remplacez les valeurs entre guillemets :

```
git config --global user.name "Prénom Nom"
git config --global user.email "prenom.nom@exemple.fr"
git config --global init.defaultBranch main
git config --global core.editor nano
git config --global pull.rebase false
```

- `user.name` et `user.email` : identité inscrite dans *chaque* commit. Utilisez la même adresse que sur votre compte GitHub ou GitLab.
- `init.defaultBranch main` : nom de la branche principale des nouveaux dépôts (anciennement

`master`).

- `core.editor nano` : éditeur que Git ouvre quand il a besoin d'un texte. Pour enregistrer et quitter `nano` : **Ctrl+O**, **Entrée**, puis **Ctrl+X**.
- `pull.rebase false` : demande à `git pull` de fusionner (*merge*), le comportement le plus simple à comprendre (voir partie 9).
- `--global` : le réglage vaut pour tous vos dépôts. Il est stocké dans le fichier `~/.gitconfig`.

Obtenir de l'aide

`git <commande> -h` affiche un résumé des options, `git help <commande>` la documentation complète.

Exercice 2 – Configurer Git

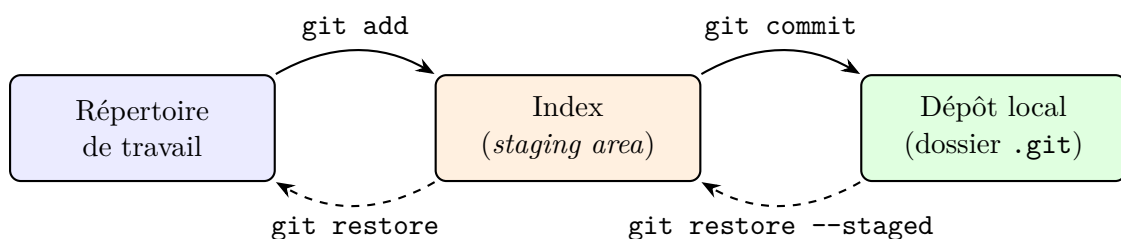
1. Affichez la version de Git installée et notez-la.
2. Appliquez la configuration ci-dessus avec *vos* informations.
3. Affichez la configuration avec `git config --global --list`, puis le contenu du fichier avec `cat ~/.gitconfig`. Que constatez-vous ?
4. Quelle commande affiche uniquement l'adresse e-mail configurée ?

Correction

1. Il faut au moins Git 2.23 pour `git switch` et `git restore`, et 2.28 pour `init.defaultBranch`. Sinon, mettre Git à jour.
3. On retrouve les mêmes informations, rangées en sections `[user]`, `[init]`, `[core]` et `[pull]`. `git config --global` ne fait qu'écrire dans ce simple fichier texte.
4. `git config user.email`

3 Les trois zones de Git

Git n'enregistre rien automatiquement : vous choisissez explicitement ce qui entre dans un commit, en deux temps.



1. **Répertoire de travail** : les fichiers tels que vous les voyez et les modifiez.
2. **Index** : `git add` y dépose les modifications à inclure dans le prochain commit. C'est comme préparer un colis avant de l'expédier.
3. **Dépôt** : `git commit` enregistre le contenu de l'index dans l'historique.

Un fichier peut donc être dans l'un des états suivants, que la commande `git status` indique :

non suivi (<i>untracked</i>)	Git ne connaît pas encore ce fichier.
modifié (<i>modified</i>)	Le fichier diffère de la dernière version enregistrée, mais la modification n'est pas indexée.
indexé (<i>staged</i>)	La modification est prête à entrer dans le prochain commit.
validé (<i>committed</i>)	Le fichier est identique à la dernière version enregistrée.

Le réflexe à prendre

Tapez `git status` avant et après chaque commande. C'est la commande la plus utile de Git : elle dit où vous en êtes et suggère souvent la commande suivante.

4 Créer son premier dépôt

Fil rouge du TP : vous documentez le réseau de la salle de TP B204 (plan d'adressage, inventaire du matériel, README). Tous les fichiers sont de simples fichiers texte.

```
mkdir ~/tp-git
cd ~/tp-git
git init
ls -a
git status
```

`git init` crée le sous-dossier caché `.git`, qui contient tout l'historique et la configuration du dépôt.

Attention

- Ne modifiez jamais le contenu de `.git` à la main. Le supprimer, c'est perdre tout l'historique.
- Ne créez pas de dépôt dans un dossier qui est déjà dans un dépôt. En particulier, ne lancez pas `git init` directement dans votre dossier personnel.

Exercice 3 – Premier commit

1. Exécutez les commandes ci-dessus. Quel message affiche `git init`? Que montre `ls -a`?
2. Créez un fichier `README.md` et observez son état :

```
echo "# Documentation du réseau de la salle B204" > README.md
git status
```

Dans quelle rubrique apparaît `README.md`?

3. Indexez le fichier puis observez :

```
git add README.md
git status
```

Dans quelle rubrique apparaît-il maintenant?

4. Validez :

```
git commit -m "Ajout du README"
git status
```

Que signifie le message affiché par `git status`?

Correction

1. `Initialized empty Git repository in .../tp-git/.git/` (en français : « Dépôt Git vide initialisé dans ... »). `ls -a` montre `.`, `..` et `.git`. `git status` indique qu'il n'y a encore aucun commit.
2. *Untracked files* (fichiers non suivis).
3. *Changes to be committed* (modifications qui seront validées), avec *new file* : `README.md`.
4. Le commit affiche `[main (root-commit) a1b2c3d] Ajout du README`. Ensuite, `git status` affiche *nothing to commit, working tree clean* (rien à valider, la copie de travail est propre) : le répertoire de travail est identique au dernier commit.

Exercice 4 – Le cycle modifier, indexer, valider

1. Avec `nano` `plan_adressage.txt`, créez le fichier suivant :

```
# Plan d'adressage - salle B204
VLAN 10  ADMIN    192.168.10.0/24  passerelle 192.168.10.254
VLAN 20  ELEVES   192.168.20.0/24  passerelle 192.168.20.254
```

2. Indexez et validez ce fichier avec le message « Ajout du plan d'adressage ».
3. Ajoutez une ligne au README (attention : `>>` ajoute à la fin du fichier, `>` l'écrase) :

```
echo "Responsable : Prénom Nom" >> README.md
```

4. Lancez `git status`. Dans quel état est `README.md` ?
5. Enregistrez cette modification dans un nouveau commit, avec un message pertinent.
6. Combien de commits contient le dépôt ? Vérifiez avec `git log --oneline`.

Correction

2. `git add plan_adressage.txt` puis `git commit -m "Ajout du plan d'adressage"`.
4. Modifié (*modified*), dans la rubrique *Changes not staged for commit*.
5. `git add README.md` puis, par exemple, `git commit -m "Ajout du responsable dans le README"`.
6. Trois commits, affichés du plus récent au plus ancien.

Indexer plusieurs fichiers

`git add .` indexe tout le contenu du dossier courant. C'est pratique, mais on risque d'embarquer un fichier non désiré (brouillon, capture, mot de passe). Vérifiez toujours avec `git status` avant de valider.

5 Explorer l'historique

Commande	Rôle
<code>git log</code>	historique détaillé (touche q pour quitter)
<code>git log --oneline</code>	une ligne par commit
<code>git log --oneline --graph --all</code>	historique avec toutes les branches (utile en partie 8)
<code>git show <id></code>	détail d'un commit : auteur, date, message, différences
<code>git diff</code>	différences entre répertoire de travail et index (ce qui n'est pas indexé)
<code>git diff --staged</code>	différences entre index et dernier commit (ce qui va être validé)

Identifiant d'un commit

Chaque commit est identifié par une empreinte (*hash*) de 40 caractères hexadécimaux, calculée à partir de son contenu, par exemple `3f2a9c1e...`. Les 7 premiers caractères suffisent en général pour le désigner.

Exercice 5 – Lire les différences

- Ajoutez cette ligne à la fin de `plan_adressage.txt` :

```
VLAN 30  SERVEURS 192.168.30.0/24  passerelle 192.168.30.254
```
- Sans indexer, tapez `git diff`. Comment Git signale-t-il la ligne ajoutée ?
- Tapez `git add plan_adressage.txt`, puis de nouveau `git diff`. Qu'affiche Git ? Pourquoi ?
- Tapez `git diff --staged`. Qu'observez-vous ?
- Validez avec le message « Ajout du VLAN 30 (serveurs) ». Avec `git log --oneline`, repérez l'identifiant du commit qui a créé le plan d'adressage, puis affichez-le avec `git show`. Qui en est l'auteur ?

Correction

- La ligne est précédée d'un **+** (en vert). Une ligne supprimée serait précédée d'un **-** (en rouge).
- Rien : `git diff` compare le répertoire de travail à l'index. Après `git add`, ils sont identiques.
- La ligne ajoutée réapparaît : `git diff --staged` compare l'index au dernier commit.
- L'auteur est celui défini par `user.name` et `user.email`, d'où l'importance de la configuration.

6 Écrire de bons messages de commit

L'historique n'est utile que s'il est lisible. Quelques règles :

- une ligne courte (environ 50 caractères au plus) qui résume la modification ;
- décrire *ce qui change* et, si c'est utile, *pourquoi* ;
- un commit correspond à *une* modification cohérente : ne mélangez pas « ajout du VLAN 30 » et « correction d'une faute dans le README » ;
- bannir les messages vides de sens : « modif », « test », « ok »...

Pour un message long, tapez `git commit` sans `-m` : l'éditeur s'ouvre. Écrivez le résumé sur la première ligne, laissez une ligne vide, puis donnez les détails.

Exercice 6 – Bon ou mauvais message ?

Pour chaque message, indiquez s'il est bon (B) ou mauvais (M) et justifiez en quelques mots.

Message	B/M	Justification
1 « modif »		
2 « Ajout du VLAN 30 (serveurs) au plan d'adressage »		
3 « Correction passerelle VLAN 20 : .254 au lieu de .1 »		
4 « trucs du mardi »		
5 « Ajout inventaire, correction README, changement plan, suppression tests »		
6 « Suppression du mot de passe laissé dans config.txt »		

Correction

1. M : ne dit rien de la modification.
2. B : précis, on sait ce qui a changé.
3. B : précis, et le « pourquoi » est explicite.
4. M : aucune information exploitable.
5. M : quatre modifications sans rapport. Il fallait faire quatre commits.
6. B pour le message, mais il signale un problème de sécurité : le mot de passe *reste dans l'historique* (visible avec `git show`). Il doit être considéré comme compromis et changé (voir partie 7).

7 Annuler et corriger

Situation	Commande	Effet
J'ai modifié un fichier et je veux abandonner ces modifications (non indexées).	<code>git restore fichier</code>	Remet le fichier dans l'état de l'index. Les modifications sont perdues.
J'ai indexé un fichier par erreur.	<code>git restore --staged fichier</code>	Retire le fichier de l'index. Les modifications restent dans le répertoire de travail.
Le message de mon dernier commit est faux, ou j'ai oublié un fichier (commit pas encore partagé).	<code>git commit --amend</code>	Remplace le dernier commit par un nouveau.
Un commit, même déjà partagé, a introduit une erreur.	<code>git revert <id></code>	Crée un <i>nouveau</i> commit qui annule l'effet de l'ancien. L'historique est conservé.
Je veux voir une ancienne version d'un fichier.	<code>git show <id>:fichier</code>	Affiche le fichier tel qu'il était dans ce commit.

Attention

`git commit --amend` réécrit l'historique. Ne l'utilisez jamais sur un commit déjà envoyé (`git push`) vers un dépôt partagé. Vous croiserez aussi `git reset --hard` dans des tutoriels : cette commande est puissante mais destructrice, et elle n'est pas utilisée dans ce TP.

Exercice 7 – Réparer ses erreurs

a) « J'ai tout écrasé ! »

```
echo "oups" > plan_adressage.txt
cat plan_adressage.txt
git status
git restore plan_adressage.txt
cat plan_adressage.txt
```

Qu'est devenu le fichier ? L'aurait-on récupéré s'il n'avait jamais été indexé ni validé ?

b) Indexation par erreur

```
echo "idées en vrac" > brouillon.txt
git add brouillon.txt
git status
```

Retirez `brouillon.txt` de l'index sans le supprimer du disque. Quelle commande utilisez-vous ? Supprimez ensuite le brouillon avec `rm brouillon.txt`.

c) Faute dans le message

Créez `inventaire.csv` :

```
nom;type;ip;salle
SW-B204;switch;192.168.10.2;B204
PC-B204-01;poste;192.168.20.11;B204
```

Indexez-le et validez-le avec le message "`Ajout inventiare`" (faute volontaire). Notez l'identifiant du commit, puis corrigez :

```
git commit --amend -m "Ajout de l'inventaire du matériel"
git log --oneline
```

L'identifiant du commit a-t-il changé ? Pourquoi ?

d) Annuler un commit

Dans le plan d'adressage, remplacez la passerelle du VLAN 10 par **192.168.10.1**, puis validez avec le message « Changement de la passerelle du VLAN 10 ». C'était une erreur : annulez ce commit.

```
git revert --no-edit HEAD
git log --oneline
cat plan_adressage.txt
```

HEAD désigne ici le dernier commit de la branche courante. Combien de commits l'opération a-t-elle ajoutés ? Pourquoi préférer **revert** à **--amend** quand le commit a déjà été partagé ?

Correction

- a) Le fichier retrouve le contenu de l'index, qui est ici celui du dernier commit. S'il n'avait jamais été indexé ni validé, Git n'en aurait aucune copie : le contenu serait perdu. Morale : validez souvent.
- b) **git restore --staged brouillon.txt**. Le fichier redevient « non suivi ».
- c) Oui. **--amend** ne modifie pas le commit : il le remplace par un nouveau, dont l'empreinte est différente (elle dépend du contenu, du message, de la date...). Ceux qui auraient récupéré l'ancien commit auraient un historique incohérent avec le vôtre.
- d) Un seul, intitulé **Revert "Changement de la passerelle du VLAN 10"**, qui s'ajoute au commit erroné. La passerelle revient à **.254**. **revert** n'efface rien : l'historique partagé reste cohérent pour toute l'équipe, et l'on garde la trace de l'erreur et de sa correction.

8 Ignorer des fichiers : .gitignore

Certains fichiers ne doivent pas être versionnés : fichiers temporaires, journaux, captures réseau, fichiers générés et surtout **secrets** (mots de passe, clés, jetons). Le fichier **.gitignore**, placé à la racine du dépôt, contient un motif par ligne. Git ignore alors les fichiers *non suivis* qui correspondent à ces motifs.

Cybersécurité

Tout ce qui a été validé reste dans l'historique, même si le fichier est supprimé ensuite. Un secret poussé sur un dépôt distant doit être considéré comme **compromis**. Des robots parcourent en permanence les dépôts publics à la recherche de mots de passe et de clés.

Exercice 8 – Mettre en place un .gitignore

Créez quelques fichiers indésirables :

```
echo "mdp_admin=Azerty123" > secrets.txt
touch capture.pcap debug.log
git status
```

Créez ensuite le fichier **.gitignore** avec **nano** :

```
# Fichiers sensibles
secrets.txt
```

```
# Captures réseau et journaux
*.pcap
*.log
```

1. Relancez `git status`. Quels fichiers apparaissent encore ?
2. Faut-il versionner le fichier `.gitignore` lui-même ? Justifiez, puis faites-le.
3. Un camarade a validé par erreur `config_routeur.txt`, qui contient le mot de passe *enable* du routeur, puis a ajouté ce fichier au `.gitignore`. Le problème est-il réglé ? Que faut-il faire ?

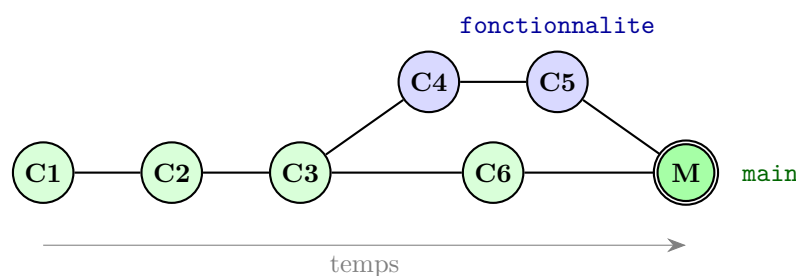
Correction

1. Seul `.gitignore` apparaît, comme fichier non suivi.
2. Oui : il fait partie du projet et doit être partagé avec l'équipe pour que tout le monde ignore les mêmes fichiers. `git add .gitignore` puis `git commit -m "Ajout du .gitignore"`.
3. Non. `.gitignore` n'agit que sur les fichiers *non suivis* : un fichier déjà validé reste suivi, et l'ancienne version reste dans l'historique (et sur le dépôt distant s'il a été poussé). Il faut :
 - arrêter le suivi sans supprimer le fichier local : `git rm --cached config_routeur.txt`, puis valider ;
 - **surtout**, considérer le mot de passe comme compromis et le changer sur le routeur.

Purger l'historique est possible (outil `git filter-repo`), mais c'est délicat et inutile si le dépôt a déjà été cloné ou publié.

9 Travailler avec des branches

Une **branche** est une ligne de développement parallèle. On l'utilise pour développer une fonctionnalité ou tester une idée sans perturber la version stable, conservée sur `main`. Une fois le travail terminé, on **fusionne** (*merge*) la branche dans `main`.



Ici, la branche `fonctionnalite` part de C3. Pendant qu'on y ajoute C4 et C5, `main` continue avec C6. Le commit de fusion M réunit les deux lignes.

<code>git branch</code>	liste les branches (la branche courante est marquée d'un <code>*</code>)
<code>git switch -c nom</code>	crée la branche <code>nom</code> et bascule dessus
<code>git switch nom</code>	bascule sur une branche existante
<code>git merge nom</code>	fusionne la branche <code>nom</code> dans la branche courante
<code>git branch -d nom</code>	supprime une branche déjà fusionnée

switch ou checkout ?

Dans beaucoup de tutoriels, vous verrez `git checkout -b nom`, qui équivaut à `git switch -c nom`. `git switch` (Git $\geq$ 2.23) est plus simple et moins ambigu. Validez ou annulez vos modifications en cours avant de changer de branche.

Exercice 9 – Une branche pour le VLAN invités

```
git switch -c vlan-invites
git branch
```

1. Quelle branche est marquée d'une étoile ?
2. Ajoutez cette ligne au plan d'adressage, puis validez avec le message « Ajout du VLAN 40 (invités) » :

```
VLAN 40  INVITES  192.168.40.0/24  passerelle 192.168.40.254
```
3. Revenez sur `main` (`git switch main`) et affichez le plan d'adressage. Où est passé le VLAN 40 ?
4. Fusionnez : `git merge vlan-invites`. Quel mot important apparaît dans la sortie ? Que signifie-t-il ?
5. Visualisez le résultat avec `git log --oneline --graph --all`, puis supprimez la branche avec `git branch -d vlan-invites`.

Correction

1. `* vlan-invites`.
3. Le VLAN 40 n'existe que dans le commit de la branche `vlan-invites`. Quand on change de branche, Git remet le répertoire de travail dans l'état du dernier commit de la branche choisie. Rien n'est perdu.
4. *Fast-forward* (avance rapide) : `main` n'avait pas évolué depuis la création de la branche. Git a simplement avancé le pointeur `main` jusqu'au commit de `vlan-invites`, sans créer de commit de fusion.

Exercice 10 – Provoquer et résoudre un conflit

1. Créez une branche `passerelle-v2`. Sur cette branche, remplacez la passerelle du VLAN 20 par `192.168.20.1` et validez (« Passerelle VLAN 20 en .1 »).
2. Revenez sur `main`. Modifiez *la même ligne* : passerelle `192.168.20.253`, puis validez (« Passerelle VLAN 20 en .253 »).
3. Affichez `git log --oneline --graph --all` et dessinez l'allure de l'historique.
4. Tentez la fusion avec `git merge passerelle-v2`. Que répond Git ? Que dit `git status` ?
5. Ouvrez `plan_adressage.txt`. Git y a inséré des **marqueurs de conflit** :

```
<<<<<< HEAD
VLAN 20  ELEVES    192.168.20.0/24  passerelle 192.168.20.253
=====
VLAN 20  ELEVES    192.168.20.0/24  passerelle 192.168.20.1
>>>>>> passerelle-v2
```

Entre `<<<<<< HEAD` et `=====` se trouve la version de la branche courante (`main`).

Entre ===== et >>>>>> se trouve celle de la branche fusionnée.

6. **Résolvez le conflit.** La convention de l'établissement place la passerelle sur la dernière adresse utilisable du réseau. Éditez le fichier pour ne garder qu'une ligne VLAN 20, avec la bonne passerelle, et supprimez les trois lignes de marqueurs. Puis :

```
git add plan_adressage.txt
git commit -m "Fusion de passerelle-v2 : passerelle VLAN 20 fixée à .254"
git log --oneline --graph --all
```

7. Pourquoi Git n'a-t-il pas pu fusionner seul ? Dans quel cas y serait-il arrivé ?

Faire marche arrière

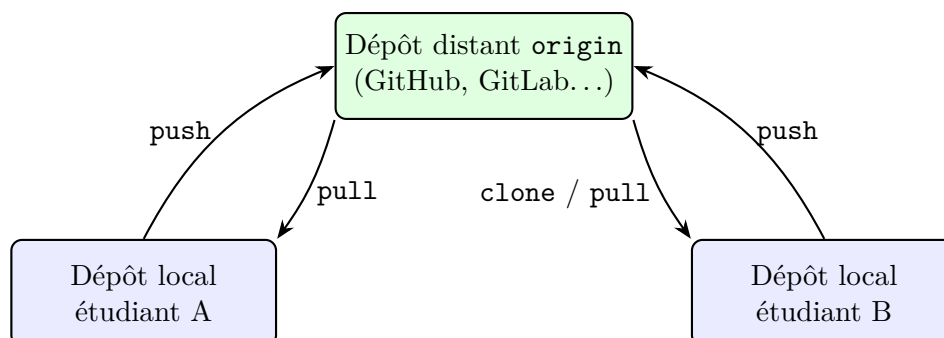
Pendant une fusion en conflit, `git merge --abort` annule la fusion et revient à l'état d'avant.

Correction

- Deux lignes qui divergent à partir du commit commun : un commit sur `passerelle-v2`, un commit sur `main`.
- CONFLICT (content): Merge conflict in `plan_adressage.txt` puis `Automatic merge failed; fix conflicts and then commit the result.`
`git status` signale *You have unmerged paths et both modified : plan_adressage.txt.*
- Aucune des deux versions n'était correcte : la bonne ligne est `VLAN 20 ELEVES 192.168.20.0/24 passerelle 192.168.20.254`. La résolution d'un conflit est une décision humaine, ici une décision technique d'administration réseau.
- Les deux branches ont modifié *la même ligne* de façon différente depuis leur ancêtre commun. Git ne peut pas savoir quelle version est la bonne. Si les modifications avaient porté sur des parties éloignées du fichier, ou sur des fichiers différents, Git aurait fusionné automatiquement.

10 Collaborer avec un dépôt distant

Pour travailler à plusieurs, on partage un **dépôt distant** hébergé sur un serveur (GitHub, GitLab, ou un serveur Gitea de l'établissement). Chacun travaille dans son dépôt local et se synchronise avec le dépôt distant.



<code>git remote add origin <url></code>	associe un dépôt distant, nommé origin , au dépôt local
<code>git remote -v</code>	liste les dépôts distants configurés
<code>git push -u origin main</code>	envoie la branche main . L'option -u mémorise le lien : ensuite, git push suffit.
<code>git pull</code>	récupère les nouveaux commits du distant et les fusionne (git fetch + git merge)
<code>git clone <url></code>	copie complète d'un dépôt distant, historique compris, avec origin déjà configuré

Authentification

Depuis 2021, GitHub refuse le mot de passe du compte pour les opérations Git en HTTPS. Sous Windows, *Git Credential Manager* (installé avec Git for Windows) ouvre une fenêtre de connexion dans le navigateur. Sous Linux, utilisez un *jeton d'accès personnel* (*personal access token*) à la place du mot de passe, ou une clé SSH.

Exercice 11 – Travailler en binôme (étudiants A et B)

1. (A) Sur GitHub ou GitLab, créez un dépôt **vide** nommé **tp-git** : sans README, sans **.gitignore** et sans licence. Sinon, l'historique distant ne correspondrait pas au vôtre.
2. (A) Reliez votre dépôt local au dépôt distant et envoyez-le (remplacez l'URL par la vôtre) :

```
git remote add origin https://github.com/compte-A/tp-git.git
git push -u origin main
```

Rafraîchissez la page web du dépôt. Que voyez-vous ?

3. (A) Ajoutez B comme collaborateur (sur GitHub : *Settings* → *Collaborators*). (B) Acceptez l'invitation.
4. (B) Clonez le dépôt de A (dans un dossier qui n'est pas déjà un dépôt Git) :

```
git clone https://github.com/compte-A/tp-git.git tp-git-A
cd tp-git-A
git log --oneline
```

L'historique est-il complet ?

5. (B) Ajoutez la ligne **SW-B204-2;switch;192.168.10.3;B204** à **inventaire.csv**, validez, puis tapez **git push**.
6. (A) Tapez **git pull** et vérifiez le contenu de l'inventaire.
7. **Travail simultané.** Sans vous synchroniser, A modifie le titre du README et B ajoute une ligne à l'inventaire. Chacun valide. B pousse en premier, puis A tente **git push**. Que se passe-t-il ? Comment A doit-il procéder ?

Correction

2. Les fichiers du projet et l'historique complet des commits.
4. Oui : `git clone` copie tout l'historique, pas seulement la dernière version.
7. Le push de A est refusé : ! [rejected] main -> main (fetch first). Le distant contient des commits que A n'a pas. A doit faire `git pull`. Les modifications portant sur des fichiers différents, la fusion est automatique. `nano` s'ouvre alors pour le message du commit de fusion : il suffit d'enregistrer et de quitter. En cas de conflit, on le résout comme dans l'exercice 10. A peut ensuite faire `git push`. Prolongement possible : faire modifier à A et B *la même ligne* pour provoquer un conflit à distance.

La règle d'or du travail en équipe

`git pull` avant de commencer à travailler ; des commits petits et fréquents ; `git push` dès qu'un travail cohérent est validé.

11 Défi de synthèse (en autonomie, 30 min)

Vous rédigez la procédure de sauvegarde de la configuration des switches de la salle. Essayez de réaliser ce défi *sans relire les pages précédentes*, en vous aidant uniquement de l'aide-mémoire.

1. Créez un nouveau dépôt `defi-git`, en dehors du dossier `tp-git`.
2. Créez `procedure.md` avec un titre et trois étapes numérotées. Validez.
3. Ajoutez un `.gitignore` qui ignore les fichiers `*.tmp`. Vérifiez-le en créant `test.tmp`. Validez.
4. Créez une branche `ajout-tftp`. Ajoutez une étape « Copier la configuration vers le serveur TFTP (`copy running-config tftp:`) ». Validez, fusionnez dans `main`, puis supprimez la branche.
5. Sur `main`, ajoutez une étape erronée et validez-la. Annulez-la ensuite *sans réécrire l'historique*.
6. Provoquez puis résolvez un conflit entre `main` et une branche de votre choix.
7. **Rendu** : la sortie de `git log --oneline --graph --all` et le contenu final de `procedure.md`.

Correction

Exemple de solution (les messages peuvent varier) :

```
cd ~ && mkdir defi-git && cd defi-git && git init
nano procedure.md                # titre + 3 étapes
git add procedure.md && git commit -m "Ajout de la procédure de sauvegarde"
echo "*.tmp" > .gitignore && touch test.tmp && git status    # test.tmp absent
git add .gitignore && git commit -m "Ajout du .gitignore"
git switch -c ajout-tftp
nano procedure.md                # étape TFTP
git commit -am "Ajout de l'étape de copie TFTP"
git switch main && git merge ajout-tftp && git branch -d ajout-tftp
nano procedure.md                # étape erronée
git commit -am "Ajout d'une étape erronée"
git revert --no-edit HEAD
# conflit : même ligne modifiée sur une branche et sur main, puis git merge,
```

```
# édition du fichier, git add, git commit
git log --oneline --graph --all
```

`git commit -am` indexe et valide en une fois les fichiers *déjà suivis*. On l'accepte si l'élève la découvre, sans l'exiger. Points de vigilance à l'évaluation : au moins 7 commits, un commit *Revert*, un commit de fusion issu d'un conflit, aucun marqueur <<<<<< restant, `test.tmp` absent de l'historique.

Auto-évaluation

Je sais...	Pas encore	En partie	Oui
expliquer la différence entre Git et GitHub	☐	☐	☐
expliquer le rôle du répertoire de travail, de l'index et du dépôt	☐	☐	☐
créer un dépôt et enregistrer des commits avec des messages clairs	☐	☐	☐
lire l'historique et les différences (<code>log</code> , <code>diff</code> , <code>show</code>)	☐	☐	☐
annuler une modification ou un commit (<code>restore</code> , <code>revert</code>)	☐	☐	☐
utiliser <code>.gitignore</code> et ne jamais versionner un secret	☐	☐	☐
créer une branche, la fusionner et résoudre un conflit	☐	☐	☐
synchroniser mon travail avec un dépôt distant (<code>clone</code> , <code>pull</code> , <code>push</code>)	☐	☐	☐

12 Testez-vous

1. Quelle commande fait passer une modification du répertoire de travail à l'index ?
☐ `git commit` ☐ `git add` ☐ `git push` ☐ `git init`
2. Que signifie *nothing to commit, working tree clean* ?
3. Quelle est la différence entre `git revert` et `git commit --amend` ?
4. Que fait `git pull`, en deux étapes ?
5. Vous avez poussé un fichier contenant un mot de passe. Suffit-il de le supprimer et de valider ?

Correction

1. `git add`.
2. Aucune modification en attente : le répertoire de travail et l'index sont identiques au dernier commit.
3. `revert` ajoute un nouveau commit qui annule un commit existant, sans réécrire l'historique (utilisable après un `push`). `--amend` remplace le dernier commit et réécrit l'historique (à réserver aux commits non partagés).
4. Il récupère les commits du dépôt distant (`git fetch`) puis les fusionne dans la branche courante (`git merge`).
5. Non : le mot de passe reste dans l'historique, localement et sur le dépôt distant. Il faut le considérer comme compromis et le changer.

Aide-mémoire Git

Configuration (une seule fois par poste)

<code>git config --global user.name "Nom"</code>	définir son nom
<code>git config --global user.email "mail"</code>	définir son adresse e-mail
<code>git config --global --list</code>	afficher la configuration

Créer ou récupérer un dépôt

<code>git init</code>	transformer le dossier courant en dépôt
<code>git clone <url></code>	copier un dépôt distant

Enregistrer

<code>git status</code>	état des fichiers (<i>à utiliser sans modération</i>)
<code>git add fichier</code>	indexer une modification
<code>git commit -m "message"</code>	valider le contenu de l'index

Consulter

<code>git log --oneline --graph --all</code>	historique compact avec les branches
<code>git show <id></code>	détail d'un commit
<code>git diff / git diff --staged</code>	modifications non indexées / indexées

Annuler

<code>git restore fichier</code>	abandonner les modifications non indexées
<code>git restore --staged fichier</code>	retirer un fichier de l'index
<code>git commit --amend</code>	corriger le dernier commit (s'il n'est pas partagé)
<code>git revert <id></code>	annuler un commit par un nouveau commit

Branches

<code>git branch</code>	lister les branches
<code>git switch -c nom</code>	créer une branche et basculer dessus
<code>git switch nom</code>	changer de branche
<code>git merge nom</code>	fusionner nom dans la branche courante
<code>git merge --abort</code>	abandonner une fusion en conflit
<code>git branch -d nom</code>	supprimer une branche fusionnée

Dépôt distant

<code>git remote add origin <url></code>	associer un dépôt distant
<code>git push -u origin main</code>	premier envoi de main
<code>git push / git pull</code>	envoyer / récupérer et fusionner

Pour aller plus loin

- S. Chacon et B. Straub, *Pro Git*, 2^e éd., Apress, 2014. Libre, traduit en français : <https://git-scm.com/book/fr/v2>. Les chapitres 1 à 3 couvrent ce TP.
- *Learn Git Branching*, exercices interactifs sur les branches, disponible en français : <https://learngitbranching.js.org>.
- Documentation de référence des commandes : <https://git-scm.com/docs>.